SQL-интеграция с LLM для обработки текста

Выполнил: Слубик Станислав Олегович
Магистратура ВШЭ, Инженерия данных
2026 год

20 марта 2026 г.

1 Введение и актуальность

В рамках финального проекта разработано пользовательское расширение `pg_llm_utils` для СУБД PostgreSQL (версия 17). Актуальность работы обусловлена растущей потребностью в интеграции больших языковых моделей (LLM) и алгоритмов машинного обучения непосредственно в транзакционные базы данных (in-database AI). Традиционный подход требует создания отдельных промежуточных микросервисов для извлечения данных, их векторизации и отправки в LLM. Разработанное расширение решает эту проблему, позволяя выполнять задачи обработки естественного языка (NLP) и реализовывать конвейер Retrieval-Augmented Generation (RAG) исключительно средствами SQL-запросов.

В качестве предметной области и тестового набора данных использован публичный датасет бразильской электронной коммерции Olist (около 100 000 строк). Расширение обладает высоким потенциалом для повторного использования: предложенная архитектура семантического кэширования и вызова API универсальна и может применяться аналитиками или инженерами данных в любых других проектах для классификации текстов, анализа тональности или построения корпоративных Q&A-систем без выгрузки данных из контура СУБД.

2 Инструкция по развертыванию и запуску (Zero-to-Production)

Развертывание системы максимально автоматизировано с использованием контейнеризации. Инфраструктура автоматически разворачивает СУБД, загружает исходные данные датасета и подготавливает среду для компиляции расширения.

1. **Подготовка окружения и данных:**
   - Поместить CSV-файлы датасета Olist в директорию `./data/` в корне проекта.
   - Создать файл `.env` со следующим содержимым:

```
GROQ_API_KEY=your_token_groq_api
DB_POSTGRES=admin
DB_PASSWORD=admin

```

2. **Запуск контейнера:**

   Выполнить команду:

```
docker compose up -d --build

```

   Примечание: При первом старте Docker автоматически выполнит скрипт `init_olist.sql` (проброшенный через `/docker-entrypoint-initdb.d/`), который создаст DDL-схемы для 9 таблиц и выполнит `COPY` данных из CSV в БД.

3. **Установка расширения:**

   Зайти в контейнер и скомпилировать расширение средствами PGXS:

```
docker exec -it pg_llm_db bash
cd /pg_extension
make install

```

4. **Активация и тестирование:**

   Подключиться к БД и запустить демонстрационный скрипт:

```
psql -U admin -d olist_db
CREATE EXTENSION IF NOT EXISTS pg_llm_utils;
\i /pg_extension/demo.sql

```

3 Инфраструктура проекта

Для реализации надежной и воспроизводимой архитектуры использован следующий стек технологий:

Таблица 1: Стек технологий проекта

Компонент	Версия / Описание	Назначение
PostgreSQL	17 (Debian)	Базовое транзакционное хранилище данных
pgvector	0.4.2	Системное расширение для хранения и поиска по векторным эмбеддингам
PL/Python3u	PostgreSQL module	Процедурный язык для реализации AI-логики внутри СУБД
Python	3.13.5	Изолированная среда выполнения скриптов внутри контейнера БД
Groq SDK / API	1.1.1 / `llama-3.1-8b`	Облачный провайдер LLM со сверхбыстрым инференсом
Sentence-Transformers	3.0.4	Библиотека для локальной генерации эмбеддингов (`MiniLM`)
Docker / Compose	3.8	Оркестрация среды, проброс портов и переменных окружения

4 Состав и архитектура расширения

Кодовая база расширения организована по стандартам PostgreSQL (файлы `pg_llm_utils.control`, `Makefile`, `pg_llm_utils-1.0.sql`). Основная логика инкапсулирована в следующих объектах БД:

Таблицы:
- `llm_cache` — реализует механизм кэширования ответов LLM на основе MD5-хэша промпта. Снижает задержки и экономит квоты API.
- `review_embeddings` — хранилище для текстов отзывов и их векторных представлений (тип `vector(384)`).

Функции и процедуры:
- `groq_chat(prompt, model)` — осуществляет HTTP-вызов к API Groq через библиотеку Python `requests/groq`. Содержит логику проверки локального кэша БД с использованием модуля `plpy`.
- `generate_embedding(text)` — локально преобразует текст в числовой вектор. Модель машинного обучения загружается непосредственно в оперативную память (RAM) СУБД с помощью глобального словаря `GD` (Global Dictionary), что исключает накладные расходы на инициализацию при массовой обработке.
- `vectorize_reviews(limit)` — хранимая PL/pgSQL процедура для пакетного расчета эмбеддингов по историческим данным.
- `ask_olist(question)` — основная функция бизнес-логики (RAG-пайплайн).

5 Математические и алгоритмические методы

В ядре системы применяются методы обработки естественного языка и вычислительной геометрии:

Векторные представления (Text Embeddings): Метод преобразования семантики текста в плотное векторное пространство непрерывных чисел $\mathbb{R}^n$. В проекте используется мультиязычная модель-трансформер, кодирующая как португальские, так и русские тексты в единое пространство размерностью $n = 384$.

Косинусное сходство (Cosine Similarity): Метрика близости между двумя векторами в многомерном пространстве. Определяется как косинус угла между ними:

$$\text{similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} \tag{1}$$

В СУБД данный математический аппарат аппаратно ускорен оператором `<=>` расширения `pgvector`. Чем меньше косинусное расстояние, тем ближе по смыслу отзывы в базе ($\mathbf{A}$) к вопросу пользователя ($\mathbf{B}$).

Retrieval-Augmented Generation (RAG): Алгоритм преодоления галлюцинаций LLM. Векторный поиск извлекает из БД наиболее релевантные факты (K-Nearest Neighbors), которые динамически внедряются в контекст запроса (Prompt) перед его отправкой в генеративную нейросеть.

6 Демонстрация работы

Скрипт `demo.sql` демонстрирует ключевые возможности расширения.

```
olist_db=# -- 1. Активация расширений
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS plpython3u;
DROP EXTENSION IF EXISTS pg_llm_utils CASCADE;
CREATE EXTENSION pg_llm_utils;
NOTICE:  extension "vector" already exists, skipping
CREATE EXTENSION
NOTICE:  extension "plpython3u" already exists, skipping
CREATE EXTENSION
DROP EXTENSION
CREATE EXTENSION
```

Рис. 1: Демонстрация работы расширения

```
olist_db=# -- 2. Проверка таблиц и базовой статистики Olist (Подтверждение загрузки данных)
SELECT count(*) AS total_reviews FROM olist_order_reviews;
SELECT count(*) AS empty_reviews FROM olist_order_reviews WHERE review_comment_message IS NULL;
 total_reviews
---------------
         98410
(1 row)

 empty_reviews
---------------
         57742
(1 row)
```

Рис. 2: Демонстрация работы расширения (продолжение)

```
olist_db=# \dt
                    List of relations
 Schema |                Name                | Type  | Owner
--------+------------------------------------+-------+-------
 public | llm_cache                          | table | admin
 public | llm_settings                       | table | admin
 public | olist_customers                    | table | admin
 public | olist_geolocation                  | table | admin
 public | olist_order_items                  | table | admin
 public | olist_order_payments               | table | admin
 public | olist_order_reviews                | table | admin
 public | olist_orders                       | table | admin
 public | olist_products                     | table | admin
 public | olist_sellers                      | table | admin
 public | product_category_name_translation  | table | admin
 public | review_embeddings                  | table | admin
(12 rows)
```

Рис. 3: Демонстрация работы расширения (окончание)

6.1 Пример 1: Интеллектуальное кэширование запросов

При вызове функции `groq_chat` система обращается к API и переводит сложный отзыв. При повторном выполнении идентичного запроса ответ возвращается мгновенно, а СУБД выдает системное уведомление `NOTICE: Groq API: Результат взят из кэша (хэш: ...)`.

```
olist_db=# -- 3. ДЕМОНСТРАЦИЯ ФУНКЦИИ 1: Семантическое кэширование и прямой запрос к LLM
-- Выполняем задачу: Перевод и саммаризация сложного бразильского отзыва на лету
-- Первый запуск (выполняет API-вызов к Groq - Llama-3)
SELECT
    review_score,
    review_comment_message AS original_portuguese,
    groq_chat('You are an expert. Please translate to Russian and extract the main problem: ' ||
review_comment_message) AS ai_analysis
FROM olist_order_reviews
WHERE review_score = 1 AND review_comment_message IS NOT NULL
LIMIT 1;
NOTICE:  Groq API: Результат взят из кэша (хэш: 93c25f1a)
```

Рис. 4: Интеллектуальное кэширование запросов (первый вызов)

```
 review_score | original_portuguese |                                                
```

```
          prompt_hash            |      model_name      |           cached_response            |         created_at
-----------------------------------+----------------------+--------------------------------------+----------------------------
 93c25f1af2ea3130893765be1d7c01ea | llama-3.1-8b-instant | Предложение переведённое на русский:+| 2026-03-19 20:29:45.428901
                                  |                      |                                     +|
                                  |                      | Пессимозно                          +|
                                  |                      |                                     +|
                                  |                      | ...                                  |
(1 row)
```

Рис. 6: Интеллектуальное кэширование запросов (уведомление о кэше)

6.2 Пример 2: In-Database векторизация

Функция `generate_embedding` инициализирует ML-модель в памяти сервера БД и возвращает математический массив на 384 размерности напрямую в SQL-вывод.

```
olist_db=# -- 4. ДЕМОНСТРАЦИЯ ФУНКЦИИ 2: Генерация эмбеддингов локально в RAM PostgreSQL (sentence-t
ransformers)
-- Первая генерация подгрузит модель 'paraphrase-multilingual-MiniLM-L12-v2' в оперативную память
SELECT generate_embedding('Где моя посылка? Доставка задерживается.') AS vector_example;
NOTICE:  Загрузка ML модели: paraphrase-multilingual-MiniLM-L12-v2 ...
NOTICE:  ML Модель успешно загружена в RAM сервера СУБД.
```

Рис. 7: In-Database векторизация (пример 1)

```
vector_example | [0.011205488,-0.05476352,0.01611325,-0.0044766967,0.3003493,-0.14827394,0.100317955
,-0.06882717,0.018418767,-0.018193878,0.018058963,0.05641316,-0.058008716,-0.11216127,-0.20674695,0.
10977794,0.01944405,-0.18554825,-0.006623515,-0.018353654,-0.050062153,-0.119302206,-0.016916517,0.1
4835398,0.1543889,0.2103734,-0.03513277,0.10599768,-0.11548166,-0.365423,0.09002059,-0.15607913,-0.1
5132804,0.09704541,0.45494983,-0.038491014,0.07304392,-0.115988426,0.13904876,-0.06103432,0.27563697
,-0.026394837,-0.0015953052,0.079278775,0.10460447,0.02762868,0.16479962,0.08598132,0.093158446,0.08
028735,0.07814969,0.15579003,-0.19889593,0.18718971,-0.067467295,0.09372551,0.20053743,0.13090307,0.
010016118,0.2449862,0.0688152,-0.045110807,-0.2762358,0.10566452,-0.4053099,-0.0151401395,-0.3142295
,-0.15723246,-0.12405074,-0.16972023,-0.045207575,-0.054253038,-0.39490476,0.15266356,0.26159427,0.0
21167384,0.18893448,0.036719978,0.04655298,0.106693216,-0.034325495,-0.0501815,-0.10873948,0.0980665
76,-0.16107967,0.21206948,0.02518446,0.36404917,-0.06417393,-0.20367256,0.31417102,-0.34159583,0.187
15452,0.088048816,-0.0067128027,-0.08705074,0.0467472,0.01663048,-0.059945427,0.4215417,0.22865666,0
.03276429,-0.028849972,-0.09677267,-0.17679106,0.003278619,0.07643123,0.06010745,0.022293957,-0.1954
4241,-0.15584102,-0.010073353,-0.21247976,0.017284777,-0.15812723,0.24742335,-0.28013617,0.10532809,
-0.029534262,-0.023378167,-0.17215045,-0.006268452,0.26314458,-0.23526946,-0.2172373,-0.297119944,0.1
9549833,0.061166026,-0.08542811,0.1103443,-0.1125696,0.21369736,-0.07624261,-0.17347322,0.116416804,
0.1167763,-0.12279428,0.05028919,-0.0025457214,0.033787068,-0.10048979,0.11513289,-0.21508318,0.0235
99366,0.15828973,0.008878896,0.34315428,-0.113416195,0.050917175,0.010354271,-0.20829974,0.008538291
,0.1910186,0.062190633,-0.20546858,0.20690155,0.1365063,0.0017336048,0.044628628,0.053857174,0.08807
985,-0.25503057,0.114690796,0.087721586,-0.16712843,-0.055372257,-0.063649885,-0.145603,0.09681411,0
.04315106,-0.15114169,0.027678205,0.0017817863,0.044016454,-0.03743568,-0.072732955,0.24703941,0.286
76254,-0.23089221,-0.2954395,-0.026129004,-0.09528442,-0.27725747,-0.085801534,0.045208845,0.0606870
```

Рис. 8: In-Database векторизация (пример 2)

```
olist_db=# -- 5. ДЕМОНСТРАЦИЯ ФУНКЦИИ 3: RAG (Retrieval-Augmented Generation) и Векторный Поиск (pgv
ector)
-- Векторизуем 500 случайных текстовых отзывов из датасета (займет 1-2 минуты в зависимости от CPU)
CALL vectorize_reviews(500);
NOTICE:  Успешно векторизовано отзывов в этой пачке: 500
CALL
```

Рис. 9: In-Database векторизация (пример 3)

```
olist_db=# -- Проверяем, что отзывы векторизовались
SELECT count(*) as total_vectors FROM review_embeddings;
-[ RECORD 1 ]-+----
total_vectors | 500
```

Рис. 10: In-Database векторизация (пример 4)

6.3 Пример 3: Кросс-языковой семантический RAG-поиск

Выполняется вызов:

```
SELECT ask_olist('На что чаще всего жалуются клиенту при низких оценках?');
```

СУБД векторизует вопрос на русском, находит через математический индекс отзывы бразильцев на португальском языке (например, “baixa qualidade” — низкое качество), агрегирует их, и Llama-3 возвращает аналитический структурированный ответ на русском языке.

```
olist_db=# -- Задаем естественный вопрос на русском языке к бразильск
ой базе.
-- Наша функция:
-- 1. Сгенерирует вектор для вопроса (RU).
-- 2. Найдет через pgvector <=> ближайшие отзывы по смыслу (PT).
-- 3. Сформирует промпт с контекстом.
-- 4. Получит ответ от Groq LLM.
SELECT ask_olist('На что чаще всего жалуются клиенты, если поставить
низкую оценку? Дай развернутый ответ.');
NOTICE:  Groq API: Запрос успешно выполнен и сохранен в кэш.
-[ RECORD 1 ]-----------------------------------------------------
---------------------------------------------------------------------
-----------------------------------------
ask_olist | Пользователи, давшие низкую оценку (1 или 2 балла), чаще
всего жалуются на следующее:
                                          +
          |

                                          +
          | - Нехватку пунктуальности при обслуживании клиентов. Это
очевидно из первого отзыва, в котором клиент подчеркивает важность пу
нктуальности в обслуживании.              +
          | - Низкую качество предоставляемых продуктов. Это подтверж
дается вторым отзывом, в котором клиент упоминает низкую качество про
дукта, который им был поставлен в продажу.
```

Рис. 11: Кросс-языковой семантический RAG-поиск (запрос)

```
olist_db=# SELECT ask_olist('Какие самые пр
иятные слова пишут пользователи при высоких
 оценках доставки?');
NOTICE:  Groq API: Запрос успешно выполнен
и сохранен в кэш.
-[ RECORD 1 ]-----------------------------
-----------------------------------------
-----------------------------------------
-----------------------------------------
-----------------------------------------
-----------------------------------------
-----------------------------------------
-----------------------------------------
-----------------------------------------
-----------------------------------------
-----------------------------------------
---------------------------
ask_olist | У нас есть всего один отзыв со
скорой оценкой 5. Но в этом отзыве нет прям
ого упоминания приятных слов, которые польз
ователи пишут при получении высоких оценок
доставки. Однако я могу сказать, что в этом
 отзыве есть положительная фраза: "Cada vez
 que compro mais fico satisfeita parabéns p
ela honestidade com seus clientes". Это гов
орит о том, что пользователь остались довол
ьны покупкой и считают нашу онлайн- площадк
у предлагая честную работу.
```

Рис. 12: Кросс-языковой семантический RAG-поиск (ответ)

7 Выводы

В ходе выполнения финальной работы спроектировано и реализовано полноценное расширение для PostgreSQL. Требования к объему кода, самостоятельности метода и практической применимости полностью выполнены.

Решена нетривиальная инженерная задача объединения реляционного движка с возможностями векторного поиска (pgvector) и вызовами внешних LLM-сервисов с локальным кэшированием (PL/Python). Разработанный RAG-конвейер позволяет проводить сложную кросс-языковую аналитику текстовых массивов напрямую из SQL-консоли. Проект имеет высокую модульность, легко переносится на другие серверы благодаря Docker и готов к интеграции в реальные ETL-процессы или BI-системы как законченный Data Engineering продукт.